

Descripción de la Arquitectura del Software (SAD)

Sistema Integral de Gestión de Eventos

Equipo Hexacore

Versión: 1.1

Fecha de entrega: 27/08/2026

Autores:

Daniel Cristancho

Samuel Emperador

Sebastián Sánchez

Diego Coronado

Materia: Arquitectura de Software

Historial de Cambios

Fecha	Descripción del cambio	Persona(s)
25/08/2026	Creación de la versión 1.0 del documento, consolidando el trabajo ya elaborado en <i>ArchitecturalProposal.tex</i> (atributos de calidad y propuesta arquitectónica), <i>C4Diagrams.tex</i> (los seis diagramas C4), <i>Submission/DistribucionCasosUso.pdf</i> (los 32 casos de uso) y <i>BitacoraArquitectura.md</i> (decisiones, alternativas y riesgos). Se añade además un primer Árbol de Utilidad (ASR) y los ADR de las decisiones fundacionales, que hasta ahora solo existían en la bitácora.	Equipo Hexacore (vía asistente)
27/08/2026	Versión 1.1: se cierran los puntos pendientes de la v1.0 — diagrama de clases UML del modelo de dominio, vista de componentes de los contenedores Personal, Eventos/Emergencias y Parqueaderos, ambientes de desarrollo y pruebas en la vista física, y modelo de datos detallado por microservicio.	Equipo Hexacore (vía asistente)
28/08/2026	Versión 1.2: durante la construcción del prototipo de la app móvil (Entrega 2) se decide unificar App Móvil del Cliente y App Móvil del Personal en un único contenedor App Móvil , que identifica el rol del usuario en el login y navega al conjunto de pantallas correspondiente (ver ADR-07). Se actualiza la vista de contenedores (de cuatro a tres interfaces desplegables) y ADR-02/ADR-05 en consecuencia.	Equipo Hexacore (vía asistente)

Tabla de Contenidos

Historial de Cambios	1
1 Introducción	3
2 Visión general de los requisitos funcionales	3
3 Requisitos No Funcionales (RNF)	4
4 Modelo de dominio	7
5 Stakeholders e intereses	10
6 Requisitos Arquitectónicamente Significativos (ASR)	10
7 Restricciones	16
8 Contexto y Alcance	16
9 Vista de contenedores	18
10 Vista de componentes	19
10.1 Servicio de Personal	19
10.2 Servicio de Eventos / Emergencias	19
10.3 Servicio de Parqueaderos	20
11 Vista de procesos	20
11.1 Reventa de una entrada (CU-006)	20
11.2 Evacuación ante emergencias (CU-010)	21
12 Vista física	21
13 Modelo de datos	22
14 Registros de Decisiones Arquitectónicas (ADR)	23
15 Riesgos técnicos	31
16 Glosario	32
17 Referencias	33

1. Introducción

El **Sistema Integral de Gestión de Eventos** centraliza en una sola plataforma los procesos que hoy suelen estar dispersos en la organización de un evento masivo (conciertos, festivales, partidos deportivos): venta de entradas y su mercado secundario, gestión de personal y turnos, parqueaderos, pedidos y establecimientos de alimentos, logística y operación del evento, y la administración general (usuarios, roles, recintos, proveedores, pagos y reportes). El problema central que resuelve es la **ausencia de un sistema unificado** que coordine estos procesos de forma consistente, segura y escalable, evitando información duplicada, errores humanos y falta de trazabilidad, especialmente crítico cuando hay miles de asistentes concurrentes durante la apertura de venta o el ingreso al recinto.

Los principales objetivos de negocio del sistema son: (1) permitir la venta y reventa segura de entradas sin duplicar propietarios ni sobrevender localidades; (2) mantener la disponibilidad del servicio durante los picos de tráfico propios de la apertura de venta, el ingreso masivo y las emergencias; (3) dar trazabilidad y auditoría a las operaciones financieras y administrativas; y (4) ofrecer una experiencia adecuada a cada tipo de usuario (cliente, personal operativo, administración) sin que ninguna interfaz duplique lógica de negocio.

Los atributos de calidad de mayor prioridad para la arquitectura son **consistencia, disponibilidad, seguridad y rendimiento**, seguidos de **tiempo real, escalabilidad y trazabilidad**; con prioridad baja-media se consideran **usabilidad, mantenibilidad y portabilidad**. La sección 6 detalla estos atributos en formato de Árbol de Utilidad.

Este documento sigue la estructura estándar *arc42* y se organiza así: primero se describen los requisitos funcionales y el modelo de dominio (secciones 2-4), luego los interesados y los requisitos arquitectónicamente significativos (secciones 5-6), después las restricciones del proyecto (sección 7) y las vistas arquitectónicas C4 (contexto, contenedores, componentes, procesos y física; secciones 8-12), y finalmente el modelo de datos, los registros de decisión (ADR), los riesgos técnicos, el glosario y las referencias.

2. Visión general de los requisitos funcionales

El sistema define **32 casos de uso** (CU-001 a CU-032), cada uno con un identificador único usado a lo largo de todo el documento. La especificación detallada de cada uno (actores, flujo principal, flujos alternativos y excepciones) está en la hoja de cálculo *Submission/CU_eventos_completo.xlsx*; la Tabla 2 resume los bloques temáticos en los que se agrupan.

Bloque temático	Casos de uso	Alcance funcional
Boletería / Compra de entradas	CU-001 – CU-005	Consulta de eventos, compra, validación de QR en el ingreso, cancelaciones/devoluciones y promociones.

Bloque temático	Casos de uso	Alcance funcional
Mercado secundario y personal para eventos	CU-006 – CU-010	Reventa segura de entradas (caso complejo), administración y asignación de personal, cambios de turno, registro de asistencia y gestión de evacuación ante emergencias (caso complejo).
Pedidos y establecimientos de alimentos	CU-011 – CU-015	Pedido de alimentos durante el evento, administración de menú/inventario, preparación y entrega validada por QR, y gestión de los establecimientos.
Logística y operación del evento	CU-016 – CU-020	Planificación logística, asignación de personal operativo, gestión de turno/asistencia en campo, monitoreo del evento y gestión de incidentes.
Parqueaderos	CU-021 – CU-025	Reserva, ingreso/salida de vehículos, asignación de espacios, cobro y control de ocupación.
Administración general (CRUD)	CU-026 – CU-032	Gestión de eventos, cuentas de usuario, roles y permisos, recintos y zonas, proveedores, pagos/conciliación financiera, y reportes y analítica.

Tabla 2: Bloques temáticos de los 32 casos de uso del sistema.

Los casos de uso están repartidos entre los cuatro integrantes del equipo (8 por persona); esa distribución, con nombre y descripción completa de cada CU, está documentada en *Submission/DistribucionCasosUso.pdf*. Dos casos de uso se destacan como los de mayor complejidad arquitectónica y sirven de hilo conductor en las vistas de este documento: **CU-006 (Gestión del Mercado Secundario de Entradas)** y **CU-010 (Gestionar Evacuación ante Emergencias)**.

3. Requisitos No Funcionales (RNF)

Mientras los casos de uso de la sección anterior describen *qué* hace el sistema, los requisitos no funcionales describen *con qué calidad* debe hacerlo. Cada RNF de la Tabla 3 se deriva de uno de los atributos de calidad priorizados en *ArchitecturalProposal.tex* y se expresa con una **métrica y un umbral verificable**, de modo que pueda comprobarse objetivamente si se cumple o no — un requisito redactado como “el sistema debe ser rápido” no es verificable y por lo tanto no se admite en esta tabla.

Los RNF son la entrada directa de las dos secciones siguientes: los **ASR** (sección 6) seleccionan cuáles de estos requisitos tienen impacto arquitectónico real, y el análisis de tácticas de *ArchitecturalProposal.tex* justifica con qué mecanismo se satisface cada uno.

Nota sobre los umbrales. Los valores numéricos son estimaciones razonadas del equipo a partir de los escenarios del proyecto, no mediciones sobre un sistema en producción (que aún

no existe). Los umbrales de RNF-01 y RNF-02 se respaldan con las pruebas de concepto del proyecto (véase *App/PoCs/*); el resto se validará conforme se implemente cada dominio.

ID	Requisito	Atributo	Métrica y umbral verificable
RNF-01	Ninguna entrada publicada en el mercado secundario puede venderse a dos compradores (CU-006).	Consistencia	Ventas duplicadas bajo concurrencia: 0% con 50 intentos de compra simultáneos sobre la misma publicación.
RNF-02	Al transferirse una entrada, el QR anterior queda invalidado antes de emitir el nuevo (CU-006).	Consistencia	Códigos QR antiguos aceptados en validación tras una transferencia: 0 .
RNF-03	El sistema sigue atendiendo solicitudes ante la caída de una instancia de cualquier microservicio (CU-001–005, CU-002).	Disponibilidad	Disponibilidad mensual del API Gateway $\geq$ 99,9% (máx. 43 min de indisponibilidad al mes).
RNF-04	Ante la caída de una instancia, el sistema se recupera sin intervención manual.	Disponibilidad	Tiempo hasta volver a responder el 100% de las solicitudes: $\leq$ 30 s .
RNF-05	El sistema nunca almacena datos completos de tarjetas de pago (CU-001, CU-011, CU-024).	Seguridad	Campos de tarjeta persistidos en cualquier base de datos propia: 0 ; el cobro se delega íntegramente a la pasarela.
RNF-06	Toda petición a un microservicio pasa autenticada y autorizada por rol desde el API Gateway.	Seguridad	Endpoints de negocio alcanzables sin token válido: 0% sobre el total de rutas expuestas.
RNF-07	La consulta de disponibilidad de entradas responde con baja latencia durante la apertura de venta (CU-001).	Rendimiento	Latencia p95 $\leq$ 500 ms con 2.000 usuarios concurrentes.
RNF-08	La validación del QR en el ingreso no genera filas perceptibles (CU-002, CU-005).	Rendimiento	Latencia p95 $\leq$ 300 ms por validación.
RNF-09	El aforo y la ocupación se reflejan casi de inmediato tras un ingreso o salida (CU-019, CU-025).	Tiempo real	Retardo entre el evento y su reflejo en el tablero de monitoreo: $\leq$ 2 s .
RNF-10	El servicio de Entradas escala horizontalmente sin degradar los demás dominios (CU-001, CU-006).	Escalabilidad	Al duplicar sus instancias, el throughput crece $\geq$ 1,7 $\times$ (eficiencia $\geq$ 85%) sin afectar la latencia de otros servicios.

ID	Requisito	Atributo	Métrica y umbral verificable
RNF-11	Toda operación financiera queda auditada (CU-006, CU-031).	Trazabilidad	Operaciones con registro completo (actor, marca de tiempo, id. de transacción, resultado): 100% ; retención ≥ 1 año.
RNF-12	Un cliente sin capacitación previa completa la compra de una entrada (CU-001).	Usabilidad	$\geq 80\%$ de éxito en prueba con 5 usuarios, en ≤ 5 pasos desde el evento hasta la confirmación.
RNF-13	Un cambio en las reglas de un dominio no obliga a modificar otro microservicio.	Mantenibilidad	Microservicios que deben modificarse y redesplegarse ante un cambio típico de negocio: 1 (más el contrato de API solo si este cambia).
RNF-14	Toda funcionalidad del rol Cliente está disponible desde web y desde móvil sobre el mismo contrato.	Portabilidad	CU del rol Cliente cubiertos por ambas interfaces consumiendo la misma API: 100% , con 0 reglas de negocio duplicadas en el cliente.
RNF-15	Una versión nueva de un microservicio se despliega sin interrumpir el servicio.	Despliegabilidad	Solicitudes fallidas durante el despliegue: 0 ; ventana de despliegue ≤ 10 min y reversión ≤ 5 min.
RNF-16	La pasarela de pagos puede sustituirse por otro proveedor sin tocar la lógica de negocio (CU-001, CU-024).	Integrabilidad	Componentes a modificar para el cambio de proveedor: 1 (solo el adaptador), 0 cambios en el servicio de Entradas.
RNF-17	Al activarse una evacuación, todo el personal en campo recibe su instrucción (CU-010).	Seguridad física (Safety)	Dispositivos que reciben la alerta: $\geq 99\%$ en ≤ 10 s desde la activación.
RNF-18	Cada microservicio puede probarse de forma aislada, sin levantar los demás.	Comprobabilidad	Cobertura de pruebas de la lógica de dominio $\geq 70\%$ y 100% de las dependencias externas sustituibles por dobles de prueba.

Tabla 3: Requisitos no funcionales, con métrica y umbral verificable por atributo de calidad.

4. Modelo de dominio

La Tabla 4 resume los conceptos del dominio identificados a partir de los 32 casos de uso. La Figura 1 presenta el diagrama de clases UML correspondiente (versión simplificada, con atributos clave y multiplicidades de las asociaciones principales); un modelo con navegabilidad, tipos de dato exactos y restricciones se refinará junto con el modelo entidad-relación de cada microservicio (sección 13) durante el prototipo.

Concepto	Explicación
Evento	Actividad que un organizador crea, con fecha, recinto, localidades, precios y aforo. Su categoría (concierto, teatro, deportes, festival, gastronomía) es un atributo del evento y es el criterio con el que el cliente filtra la cartelera.
Entrada	Boleto asociado a una localidad de un evento, con un código QR único que identifica al propietario actual.
Publicación de reventa	Oferta de una entrada ya adquirida, puesta en venta por su propietario en el mercado secundario.
Recinto / Zona	Lugar físico donde se realiza el evento y sus subdivisiones (zonas, localidades), cada una con capacidad y mapa de distribución.
Usuario / Cuenta	Persona registrada en el sistema (cliente, empleado, organizador o administrador), con credenciales y un conjunto de roles asignados.
Rol / Permiso	Conjunto de acciones autorizadas sobre un módulo del sistema, asignado a una cuenta de usuario.
Personal / Empleado	Persona que trabaja en la operación de un evento, con un rol operativo (seguridad, logística, alimentos, etc.).
Turno / Asignación	Periodo de trabajo de un empleado en un evento, con horario y zona asignada.
Registro de asistencia	Marca de ingreso y salida de un empleado durante un turno, usada para calcular horas trabajadas.
Parqueadero / Espacio	Zona de estacionamiento del evento y cada unidad de espacio reservable dentro de ella.
Pedido	Solicitud de productos de alimentos hecha por un cliente a un establecimiento durante un evento.
Establecimiento / Producto	Punto de venta de alimentos dentro del evento y los productos de su menú.
Pago	Transacción monetaria asociada a una compra, reventa, pedido o servicio de parqueadero, procesada mediante la pasarela de pagos externa.
Proveedor	Tercero externo (catering, sonido, seguridad privada, etc.) contratado para la operación de un evento.

Concepto	Explicación
Incidente / Protocolo de evacuación	Evento anómalo registrado durante la operación (desde uno menor hasta una emergencia que activa un protocolo de evacuación).
Reporte	Consulta agregada (ventas, ocupación, ingresos, personal, incidentes) usada para la toma de decisiones.

Tabla 4: Conceptos del dominio identificados a partir de los 32 casos de uso.

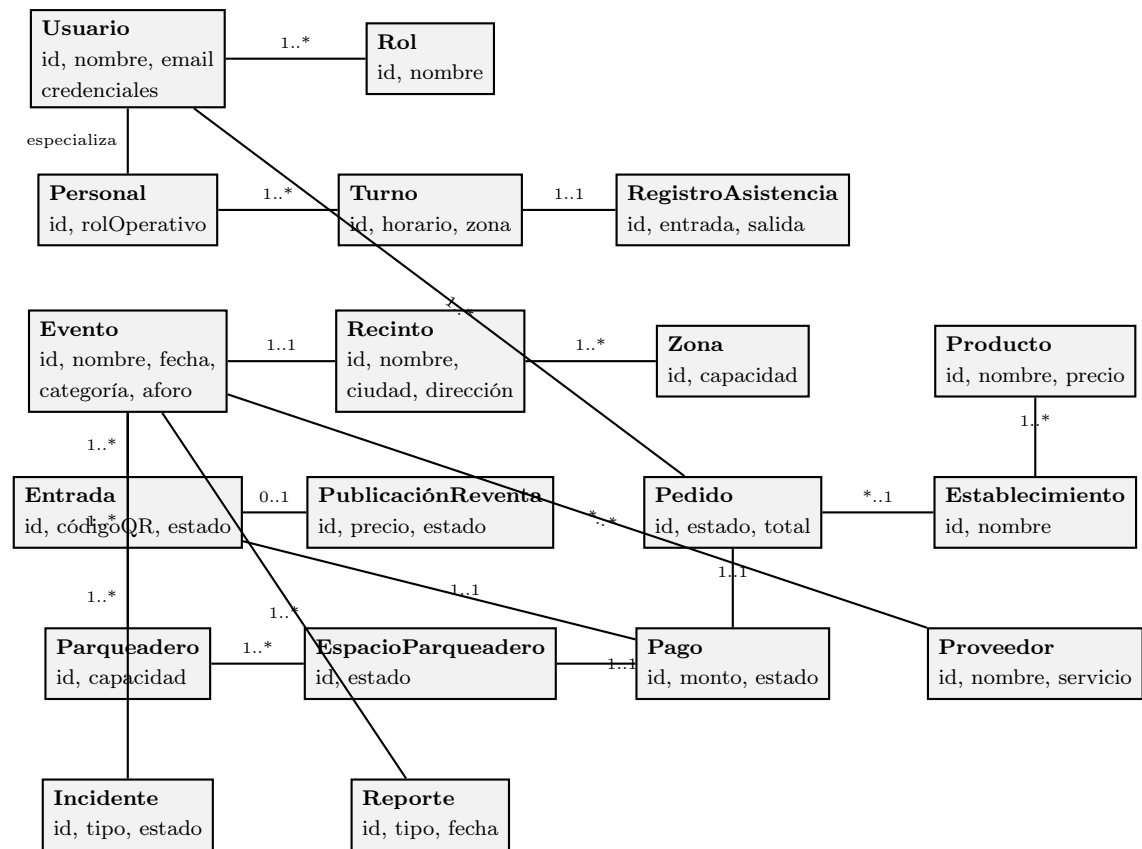


Figura 1: Diagrama de clases UML — modelo de dominio (versión simplificada).

5. Stakeholders e intereses

Rol/Nombre	Información de Contacto	Intereses / expectativas
Cliente	Rol dentro del sistema (se autentica con su cuenta)	Comprar y revender entradas de forma confiable, consultar sus pedidos y parqueadero, y recibir su QR sin fallas en el ingreso.
Organizador	Rol dentro del sistema	Crear y planificar eventos, asignar personal y proveedores, y monitorear la operación en tiempo real.
Empleado / Personal operativo	Rol dentro del sistema	Consultar sus turnos y asignaciones, solicitar cambios de turno y registrar su asistencia sin fricción.
Administrador	Rol dentro del sistema	Gestionar usuarios, roles, recintos y configuración general con control de acceso auditable.
Supervisor de emergencia	Rol dentro del sistema	Activar y coordinar protocolos de evacuación con información confiable y actualizada en tiempo real.
Proveedor externo	Gestionado por el organizador (módulo Proveedores)	Ser asignado correctamente a los eventos que contrata y recibir la información logística necesaria.
Equipo Hexacore (Daniel Cristancho, Samuel Emperador, Sebastián Sánchez, Diego Coronado)	<code>semperadorcontreras@gmail.com</code> (contacto del equipo)	Diseñar y sustentar una arquitectura que satisfaga los atributos de calidad priorizados y cumpla los requisitos del curso.
Profesor / Evaluador del curso	Curso de Arquitectura de Software	Verificar que el SAD cumpla la plantilla exigida y que las decisiones arquitectónicas estén justificadas.

6. Requisitos Arquitectónicamente Significativos (ASR)

La Tabla 6 presenta el Árbol de Utilidad del proyecto: los escenarios de calidad que tienen impacto arquitectónico real, derivados de los atributos priorizados en *ArchitecturalProposal.tex* y de los requisitos no funcionales de la sección 3. La columna **RNF** indica qué requisito verificable pone a prueba cada escenario; entre los dos elementos hay una diferencia de propósito: el RNF fija el umbral (*cuánto*), el ASR describe la situación concreta que lo tensiona (*cuándo y bajo qué condiciones*).

Cada escenario se califica con el par **(Importancia, Dificultad)** en escala Alta/Media/Baja, como pide el método del Árbol de Utilidad:

- **Importancia:** qué tanto afecta al negocio que ese escenario *no* se cumpla. Es la importancia *del escenario*, que no siempre coincide con la prioridad global de su atributo — por ejemplo, la Seguridad física está priorizada como Media en el conjunto del sistema, pero el escenario ASR-13 (fallar al notificar una evacuación) tiene importancia Alta porque compromete la integridad de las personas.
- **Dificultad:** qué tan difícil es técnicamente satisfacerlo con la arquitectura propuesta.

Los escenarios calificados **(Alta, Alta)** — ASR-01, ASR-04 y ASR-13 — son los que concentran el riesgo arquitectónico del proyecto y, por lo tanto, los candidatos naturales a respaldarse con una prueba de concepto medida en vez de solo con razonamiento.

Tabla 6: Árbol de Utilidad — escenarios de calidad con su par (Importancia, Dificultad) y el RNF que verifican.

ID ASR	Título	Atributo	Sub- atributo	CU(s)	RNF	Escenario	Impor- tancia	Difi- cultad
ASR-01	Bloqueo de doble venta en reventa	Consistencia	Concurrencia de datos	CU-006	RNF-01, RNF-02	Dos compradores intentan adquirir simultáneamente la misma entrada publicada en reventa; el sistema garantiza que solo uno complete la compra y el otro reciba de inmediato un mensaje de no disponibilidad.	Alta	Alta
ASR-02	Disponibilidad durante el ingreso masivo	Disponibilidad	Tolerancia a fallos	CU-002, CU-021, CU-022	RNF-03, RNF-04	Ante la caída de una instancia del servicio de validación de QR durante el ingreso al recinto, el balanceador redirige el tráfico a instancias sanas sin interrupción perceptible para el asistente.	Alta	Media
ASR-03	Protección de datos de pago y credenciales	Seguridad	Confidencialidad	CU-001, CU-024, CU-031	RNF-05, RNF-06	Un intento de acceso no autorizado a datos de pago o credenciales es rechazado por el API Gateway y queda registrado para auditoría.	Alta	Media

Continúa en la página siguiente. . .

Tabla 6 — continuación

ID ASR	Título	Atributo	Sub-atributo	CU(s)	RNF	Escenario	Importancia	Dificultad
ASR-04	Respuesta bajo carga en apertura de venta	Rendimiento	Tiempo de respuesta	CU-001, CU-005	RNF-07, RNF-08	Durante la apertura de venta de un evento masivo, miles de usuarios consultan disponibilidad y compran de forma simultánea; la mayoría de las solicitudes se resuelve en tiempos cortos y estables.	Alta	Alta
ASR-05	Actualización de aforo y zonas en tiempo real	Tiempo real	Actualización de estado	CU-010, CU-019	RNF-09	El aforo y el estado de las zonas se reflejan en los tableros de monitoreo con baja latencia tras el registro de un ingreso o un incidente.	Media	Alta
ASR-06	Escalado independiente del servicio de Entradas	Escalabilidad	Escalabilidad horizontal	CU-001, CU-006	RNF-10	Ante un incremento súbito de tráfico en el servicio de Entradas durante una preventa, aumentan sus instancias sin afectar el rendimiento de los demás dominios.	Media	Media
ASR-07	Auditoría de transferencias y pagos	Trazabilidad	Audita-bilidad	CU-006, CU-031	RNF-11	Cada transferencia de entrada, pago o reembolso queda registrada con usuario, fecha y resultado, consultable en un reporte de auditoría.	Media	Baja
ASR-08	Interfaces especializadas por rol	Usabilidad	Adecuación a la tarea	Todos los CU (por interfaz)	RNF-12	Un usuario nuevo de cada rol completa su tarea principal en su interfaz correspondiente sin capacitación previa.	Media	Baja

Continúa en la página siguiente...

Tabla 6 — continuación

ID ASR	Título	Atributo	Sub-atributo	CU(s)	RNF	Escenario	Importancia	Dificultad
ASR-09	Evolución independiente de dominios	Mantenibilidad	Modificabilidad	Todos los CU (por micro-servicio)	RNF-13	Un cambio en las reglas de negocio de Parqueros se implementa y despliega sin requerir cambios ni rediseño del servicio de Entradas.	Media	Media
ASR-10	Consumo uniforme vía API	Portabilidad	Independencia de plataforma	Todos los CU	RNF-14	Una misma funcionalidad de backend se consume igual desde el portal web y desde la app móvil, sin duplicar lógica de negocio.	Media	Baja
ASR-11	Sustitución del proveedor de pagos sin reescribir el dominio	Integrabilidad	Adaptación de interfaces	CU-001, CU-011, CU-024	RNF-16	La organización cambia de pasarela de pagos por costos; el equipo reemplaza el adaptador de pagos y los servicios de Entradas, Parqueros y Pedidos siguen funcionando sin modificar su lógica de negocio.	Media	Media
ASR-12	Actualización de un servicio con el evento en curso	Desplegabilidad	Despliegue sin interrupción	Todos los CU (por micro-servicio)	RNF-15	Se detecta un defecto en el servicio de Pedidos durante un evento activo; se despliega la corrección sin detener la venta ni el ingreso de asistentes, y con posibilidad de revertir si la versión nueva falla.	Media	Media

Continúa en la página siguiente...

Tabla 6 — continuación

ID ASR	Título	Atributo	Sub-atributo	CU(s)	RNF	Escenario	Importancia	Dificultad
ASR-13	Notificación garantizada de evacuación	Seguridad física (Safety)	Respuesta ante emergencia	CU-010	RNF-17	El supervisor activa el protocolo de evacuación; todo el personal en campo recibe su instrucción específica en segundos, incluso si el proveedor de notificaciones responde con lentitud o una instancia del servicio falla.	Alta	Alta
ASR-14	Prueba aislada de cada microservicio	Comprobabilidad	Aislamiento de dependencias	Todos los CU (por micro-servicio)	RNF-18	Un integrante prueba la lógica de su dominio sin levantar los otros cinco microservicios ni depender de la pasarela de pagos real, sustituyéndolos por dobles de prueba.	Media	Baja

7. Restricciones

Restricción	Categoría	Descripción
Arquitectura de microservicios con base de datos independiente por dominio	Técnica	Decisión fundacional ya tomada por el equipo (ver ADR-01); cualquier solución a un ASR debe respetar esta separación por dominio.
Soporte de picos de miles de usuarios concurrentes	Técnica	El sistema debe soportar la concurrencia propia de la apertura de venta y el ingreso masivo al recinto sin degradar el servicio.
Equipo de 4 integrantes, 8 CU por persona	Organizacional	La distribución de trabajo (Daniel Cristancho, Samuel Emperador, Sebastián Sánchez, Diego Coronado) condiciona qué componentes prototipa cada integrante para Entrega 1 y Entrega 2.
Documento basado en el estándar arc42	Convencional	La plantilla y estructura de este documento las exige el curso; no pueden omitirse secciones.
Fechas y formato de entrega del curso	Convencional	Entrega 1 (SAD v1) y Entrega 2 (prototipo + SAD v2) deben respetar las fechas y reglas de entrega definidas por el curso.

8. Contexto y Alcance

El sistema interactúa con cinco roles de usuario (Cliente, Organizador, Empleado, Administrador y Supervisor de emergencia) y dos sistemas externos (Pasarela de Pagos y Proveedor de Notificaciones). La Figura 2 muestra el diagrama de contexto (Nivel 1 del modelo C4).

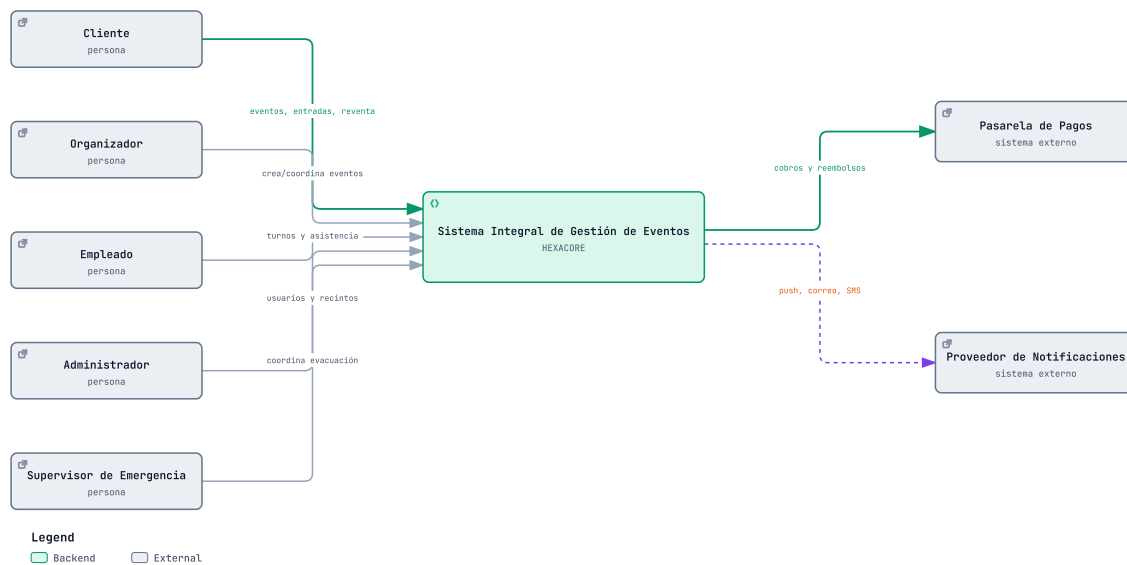


Figura 2: Diagrama de contexto (Nivel 1, modelo C4).

Origen	Destino	Descripción
Cliente	Sistema	Consulta eventos, compra y revende entradas, reserva parqueadero.
Organizador	Sistema	Crea eventos, asigna personal y coordina la operación.
Empleado	Sistema	Consulta turnos y registra ingreso/salida.
Administrador	Sistema	Administra usuarios, personal y configuración.
Supervisor de emergencia	Sistema	Activa y coordina el protocolo de evacuación.
Sistema	Pasarela de Pagos	Procesa compras, reventas y reembolsos.
Sistema	Proveedor de Notificaciones	Envía notificaciones push, correo y SMS.

Tabla 8: Relaciones del diagrama de contexto.

El diagrama de panorama de sistemas (*System Landscape, Diagrams/ArchifyPDF/04-panorama.pdf* en *C4Diagrams.tex*) amplía este alcance mostrando además cómo el sistema se relaciona con otros sistemas de la organización (Contabilidad y Facturación, CRM/Marketing y una plataforma de Analítica), fuera del alcance de este documento pero relevantes para entender el ecosistema completo.

9. Vista de contenedores

El sistema se divide en tres interfaces desplegables (Portal Web del Cliente, Portal Web Administrativo/Operativo y App Móvil), todas consumiendo el mismo backend a través de un API Gateway. La **App Móvil** está construida en **Flutter** (ADR-08), lo que permite generar desde una sola base de código los binarios de Android e iOS, y sirve tanto al rol Cliente como al rol Personal: el login identifica el rol del usuario autenticado y la navegación interna abre el conjunto de pantallas correspondiente — no son dos apps distribuidas por separado (ver ADR-07). El backend está compuesto por microservicios independientes por dominio (Entradas/Mercado Secundario, Personal, Eventos/Emergencias, Parqueaderos) y por infraestructura de soporte (Redis, cola de mensajes y bases de datos por servicio). La Figura 3 presenta el diagrama de contenedores.

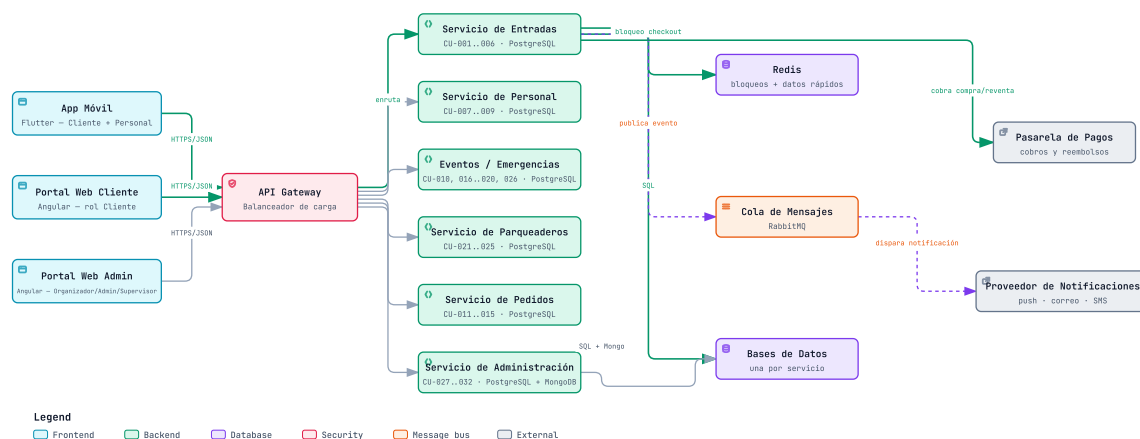


Figura 3: Diagrama de contenedores (Nivel 2, modelo C4).

- **API Gateway + balanceador de carga:** punto único de entrada al backend; centraliza autenticación, autorización, validación de tokens y enrutamiento hacia el microservicio correspondiente.
- **Microservicios:** Entradas/Mercado Secundario, Personal, Eventos/Emergencias y Parqueaderos, cada uno con su propia base de datos para reducir el acoplamiento entre dominios.
- **Redis:** bloqueo temporal de recursos en disputa (p. ej. una entrada en reventa durante el checkout) y datos de acceso rápido (aforo, ocupación, sesiones).
- **Cola de mensajes (RabbitMQ, ADR-10):** desacopla operaciones asíncronas: generación de QR, notificaciones, auditoría, liquidación de pagos y alertas de emergencia.
- **Servicios externos:** Pasarela de Pagos y Proveedor de Notificaciones, integrados vía API y, en el caso de notificaciones, disparados de forma asíncrona desde la cola de mensajes.

10. Vista de componentes

Se detalla primero el contenedor **Servicio de Entradas y Mercado Secundario**, por ser el que resuelve el caso de uso complejo CU-006 (Gestión del Mercado Secundario de Entradas), y luego, al mismo nivel de detalle, los otros tres microservicios de dominio. La Figura 4 muestra los componentes internos del Servicio de Entradas.

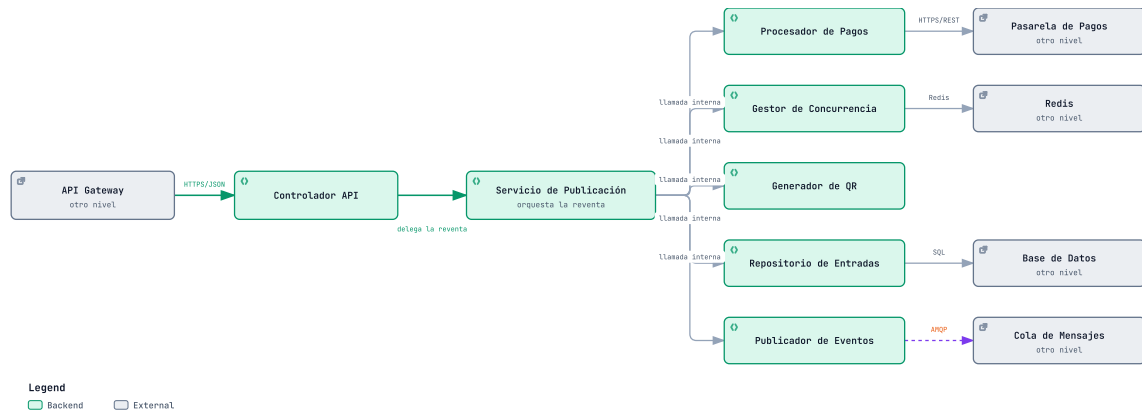


Figura 4: Diagrama de componentes — Servicio de Entradas y Mercado Secundario (Nivel 3, modelo C4).

El flujo interno es: el **Controlador API** recibe la solicitud enrutada por el API Gateway y la delega al **Servicio de Publicación**, que orquesta la operación de reventa. Este componente solicita al **Gestor de Concurrencia** el bloqueo temporal de la entrada en Redis, invoca al **Procesador de Pagos** para cobrar la transacción contra la Pasarela de Pagos, usa el **Generador de QR** para invalidar el código anterior y emitir uno nuevo, persiste el cambio de propiedad mediante el **Repositorio de Entradas**, y finalmente notifica el resultado a través del **Publicador de Eventos** hacia la cola de mensajes.

10.1. Servicio de Personal

El **Controlador API** enruta la solicitud hacia el **Servicio de Asignación de Personal**, que valida rol, disponibilidad y horario del empleado con apoyo del **Validador de Conflictos** antes de confirmar una asignación o un cambio de turno. El **Gestor de Turnos** calcula horarios y, junto con el **Repositorio de Personal**, persiste asignaciones, turnos y registros de asistencia. El **Publicador de Eventos** notifica, vía cola de mensajes, la confirmación de asignaciones y cambios de turno al empleado y al supervisor.

10.2. Servicio de Eventos / Emergencias

El **Controlador API** enruta hacia el **Servicio de Gestión de Eventos** (CRUD de eventos, localidades y aforo) o hacia el **Servicio de Protocolo de Emergencia**, según la operación. Al activarse una evacuación, este último consulta al **Gestor de Zonas y Aforo** (apoyado en

Redis para datos de baja latencia) para identificar zonas afectadas, salidas y rutas disponibles; persiste el incidente mediante el **Repositorio de Eventos**; y usa el **Publicador de Eventos** para difundir alertas y actualizaciones de estado a través de la cola de mensajes hacia el personal y los asistentes.

10.3. Servicio de Parquaderos

El **Controlador API** delega en el **Servicio de Reservas**, que gestiona la reserva previa y el ingreso/salida de vehículos. El **Gestor de Ocupación** mantiene en Redis el estado de ocupación por zona en tiempo real. El **Procesador de Cobro** invoca la Pasarela de Pagos para tarifas y reservas. El **Repositorio de Espacios** persiste las asignaciones, y el **Publicador de Eventos** notifica confirmaciones hacia la cola de mensajes.

11. Vista de procesos

Se documentan los dos procesos más críticos del sistema desde el punto de vista de los atributos de calidad: la reventa de una entrada (conurrencia y consistencia) y la evacuación ante una emergencia (disponibilidad y tiempo real).

11.1. Reventa de una entrada (CU-006)

La Figura 5 muestra, con líneas de vida y mensajes numerados, cómo colaboran los contenedores para resolver una compra en el mercado secundario.

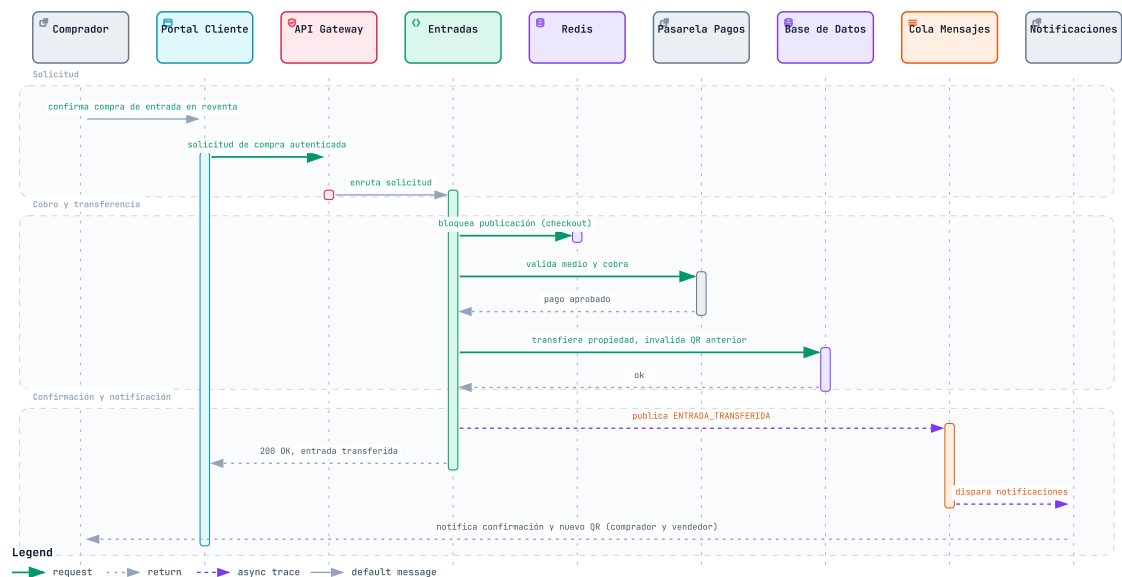


Figura 5: Diagrama de secuencia — reventa de una entrada (CU-006).

1. El comprador confirma la compra de una entrada publicada en el mercado secundario.

2. El API Gateway enruta la solicitud autenticada hacia el Servicio de Entradas.
3. El Servicio de Entradas bloquea temporalmente la publicación en Redis, para impedir que otro comprador la adquiera de forma simultánea.
4. Se valida el medio de pago y se cobra el valor de la entrada mediante la Pasarela de Pagos.
5. Se transfiere la propiedad de la entrada y se invalida el código QR anterior en la base de datos.
6. Se publica el evento `ENTRADA_TRANSFERIDA` en la cola de mensajes.
7. El Proveedor de Notificaciones envía la confirmación y el nuevo código QR al comprador y al vendedor, de forma desacoplada del flujo principal.

11.2. Evacuación ante emergencias (CU-010)

1. El supervisor activa el protocolo de evacuación desde el Portal Web Administrativo, registrando el tipo y ubicación de la emergencia.
2. El servicio de Eventos/Emergencias identifica las zonas potencialmente afectadas y consulta el aforo actual, las salidas disponibles y las rutas configuradas.
3. El sistema presenta las rutas y zonas recomendadas; el supervisor confirma el protocolo.
4. Se notifica al personal operativo y se envían instrucciones a los asistentes mediante la cola de mensajes y el Proveedor de Notificaciones.
5. El sistema monitorea las zonas en tiempo real mientras el supervisor atiende alertas (ruta bloqueada, zona congestionada, cambio de nivel de emergencia) hasta confirmar la finalización.

12. Vista física

La Figura 6 muestra el ambiente de **producción**: los dispositivos del usuario acceden vía CDN a los archivos estáticos y vía balanceador + API Gateway al backend, que corre en un clúster de aplicación con réplicas por microservicio, respaldado por un clúster de bases de datos, un clúster Redis y un clúster de mensajería.

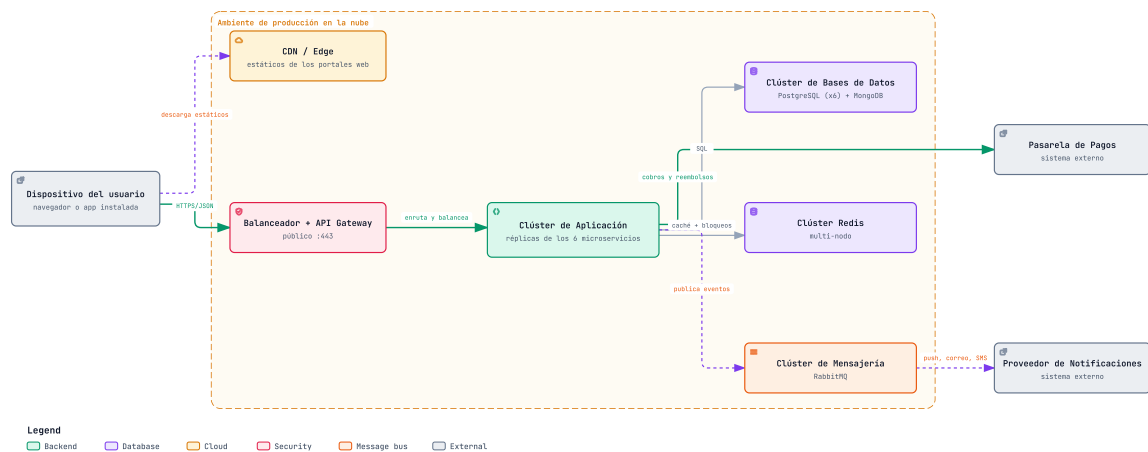


Figura 6: Diagrama de despliegue — ambiente de producción (modelo C4).

Los ambientes de **desarrollo** y **pruebas** reutilizan la misma topología del diagrama de despliegue de producción, a menor escala: una única réplica por microservicio (sin balanceo horizontal), sin CDN (los archivos estáticos se sirven directamente desde el servidor de aplicación), y con Redis, la cola de mensajes y las bases de datos desplegadas en un único nodo compartido en vez de un clúster. El ambiente de **desarrollo** se despliega automáticamente mediante integración continua ante cada cambio en la rama principal, con datos de prueba mínimos para verificación funcional rápida. El ambiente de **pruebas** se usa para validación funcional end-to-end y pruebas de carga previas a cada entrega, con datos sintéticos que simulan la concurrencia esperada durante la apertura de venta y el ingreso masivo, de forma que los ASR de disponibilidad y rendimiento (sección 6) puedan verificarse antes de producción. El script de despliegue automatizado que provisiona ambos ambientes se entregará junto con el prototipo funcional en la Entrega 2.

13. Modelo de datos

Siguiendo la decisión de una base de datos independiente por microservicio (ADR-01), cada dominio tiene su propio esquema relacional. Las referencias entre dominios (p. ej. el propietario de una *Entrada* hacia *Usuario*, que vive en el módulo de administración de cuentas) se resuelven a nivel de aplicación mediante el identificador correspondiente, y no con llaves foráneas físicas entre bases de datos distintas, lo cual es consistente con la separación de datos establecida en ADR-01. A continuación se listan las entidades principales de cada esquema, con sus llaves foráneas lógicas.

Esquema de Entradas / Mercado Secundario

- **Entrada**(id, evento_id, localidad_id, propietario_id, codigoQR, estado)
- **PublicacionReventa**(id, entrada_id, vendedor_id, precio, estado)

Esquema de Personal

- **Personal**(id, usuario_id, rolOperativo)
- **Turno**(id, personal_id, evento_id, zona_id, horario)
- **RegistroAsistencia**(id, turno_id, horaEntrada, horaSalida)

Esquema de Eventos / Emergencias

- **Evento**(id, nombre, fecha, categoria, recinto_id, aforo)
- **Recinto**(id, nombre, ciudad, direccion)
- **Zona**(id, recinto_id, capacidad)
- **Incidente**(id, evento_id, tipo, estado)

Esquema de Parqueaderos

- **Parqueadero**(id, evento_id, capacidad)
- **EspacioParqueadero**(id, parqueadero_id, estado)

Esquemas de administración general

- **Usuario**(id, nombre, email, credenciales), **Rol**(id, nombre), **UsuarioRol**(usuario_id, rol_id)
- **Proveedor**(id, nombre, servicio)
- **Pago**(id, tipo, monto, estado, referencia_id, referencia_tipo) — registra pagos de Entradas, Pedidos o Parqueaderos mediante **referencia_tipo**
- **Pedido**(id, cliente_id, establecimiento_id, estado, total), **Establecimiento**(id, nombre), **Producto**(id, establecimiento_id, nombre, precio)
- **Reporte**(id, tipo, fechaGeneracion)

El motor de base de datos concreto por dominio es **PostgreSQL** para los esquemas transaccionales y **MongoDB** para Reportes (ADR-06).

14. Registros de Decisiones Arquitectónicas (ADR)

ADR-01 — Arquitectura de microservicios por dominio

ASR relacionados: ASR-06, ASR-09.

Problema: el sistema debe permitir que sus dominios (Entradas, Mercado Secundario, Personal, Eventos, Parqueaderos, Emergencias) evolucionen y escalen de forma relativamente independiente, sin que un cambio en uno arriesgue la disponibilidad de los demás.

Solución: se adopta una arquitectura de **microservicios**, uno por dominio, cada uno con base

de datos propia.

Alternativa descartada: monolito modular — dificulta escalar de forma independiente el servicio de Entradas durante una apertura de venta sin sobre-aprovisionar los demás dominios.

Consecuencias: favorece mantenibilidad y escalabilidad independiente, a costa de mayor complejidad operativa (más piezas de infraestructura) y de tener que gestionar consistencia eventual entre servicios en vez de transacciones ACID únicas.

ADR-02 — API Gateway y balanceador de carga

ASR relacionados: ASR-02, ASR-03.

Problema: se necesita un punto único, seguro y disponible de entrada al backend para las tres interfaces del sistema.

Solución: un **API Gateway** centraliza autenticación, autorización y enrutamiento; un **balanceador de carga** distribuye solicitudes entre múltiples instancias de cada microservicio.

Alternativa descartada: exponer cada microservicio directamente a los clientes — se descartó por duplicar lógica de seguridad en cada interfaz y complicar el enrutamiento.

Consecuencias: centraliza seguridad y facilita el enrutamiento, pero introduce un componente crítico que debe replicarse (mitigado con múltiples instancias detrás del balanceador) para no convertirse en un punto único de falla.

ADR-03 — Redis para bloqueo temporal y datos de acceso rápido

ASR relacionados: ASR-01, ASR-05.

Problema: evitar que dos compradores completen simultáneamente la compra de la misma entrada en reventa, y disponer de datos de aforo/ocupación con baja latencia.

Solución: **Redis** se usa como bloqueo distribuido temporal durante el checkout de una reventa, y como caché de datos de acceso rápido (aforo, ocupación, sesiones).

Alternativa descartada: bloqueo pesimista directamente en la base de datos relacional — se descartó por convertirse en cuello de botella bajo la carga concurrente esperada.

Consecuencias: resuelve la doble venta y acelera consultas frecuentes, pero añade un punto adicional de falla que requiere su propia estrategia de disponibilidad (réplica/clúster).

ADR-04 — Cola de mensajes para operaciones asíncronas

Estado: *completado por ADR-10* — la decisión de usar una cola sigue vigente; el motor concreto, que aquí quedó abierto como “RabbitMQ/Kafka”, se cerró en ADR-10 con evidencia medida.

ASR relacionados: ASR-04, ASR-07.

Problema: operaciones como notificaciones o liquidación de pagos no deben bloquear ni hacer fallar la operación principal (compra, transferencia) cuando un servicio externo es lento o falla temporalmente.

Solución: una **cola de mensajes (RabbitMQ/Kafka)** desacopla la generación de QR, las notificaciones, la auditoría y la liquidación de pagos del flujo síncrono principal.

Alternativa descartada: comunicación síncrona directa con los servicios externos — se descartó porque un fallo temporal del proveedor de notificaciones no debe bloquear la operación principal.

Consecuencias: tolera fallos externos y mejora el rendimiento percibido, a costa de consistencia eventual y de requerir monitoreo propio de las colas.

ADR-05 — Stack de frontend por tipo de interfaz

Estado: *parcialmente superado por ADR-08* — lo relativo a los portales web sigue vigente; la decisión sobre el stack de la app móvil fue revertida el 2026-09-07. Se conserva el texto original como registro de por qué se decidió así en su momento.

ASR relacionados: ASR-08, ASR-10, ASR-05.

Problema: las tres interfaces (portal web del cliente, portal web administrativo/operativo, app móvil) deben implementarse en un lenguaje/framework concreto, sin comprometer el consumo uniforme del backend vía API Gateway (ASR-10) ni la especialización por rol de cada interfaz (ASR-08).

Solución: los portales web se construyen en **Angular + TypeScript** — su estructura de módulos, *guards* e *interceptors* favorece el trabajo en paralelo de varios integrantes con convenciones consistentes, y RxJS encaja con las actualizaciones en tiempo real de ASR-05. La app móvil se construye en **Kotlin nativo sobre Android Studio** (Jetpack Compose para la interfaz, *coroutines/Flow* para tiempo real), aprovechando que el equipo ya domina ambos lenguajes; sirve a los roles Cliente y Personal desde un mismo proyecto (ver ADR-07), por lo que ASR-08 se resuelve con navegación condicionada por rol en vez de con binarios separados.

Alternativa descartada: un framework híbrido/*cross-platform* (p. ej. React Native o Flutter) para compartir código de interfaz entre web y móvil — se descartó porque ASR-10 ya evita duplicar lógica de negocio en el cliente (esta vive en el backend, detrás del API Gateway), por lo que compartir código de UI entre plataformas no aporta un beneficio arquitectónico claro frente al costo de una capa de abstracción adicional y a la experiencia ya existente del equipo con Angular y Kotlin nativos.

Consecuencias: cada interfaz evoluciona con las herramientas nativas de su plataforma sin las limitaciones de un framework híbrido, a costa de mantener dos ecosistemas de *build/* pruebas distintos (TypeScript/npm y Kotlin/Gradle) y de no compartir código de UI entre web y móvil — los contratos comunes quedan limitados a la especificación de la API (ver App/shared/).

ADR-06 — Motor de base de datos por dominio (*polyglot persistence*)

ASR relacionados: ASR-01, ASR-07.

Problema: ADR-01 estableció una base de datos propia por microservicio, pero dejó abierto el motor concreto; los esquemas transaccionales (Entradas, Personal, Eventos/Emergencias, Parqueaderos, cuentas/proveedores/pagos) tienen necesidades de consistencia distintas a las de Reportes/analítica, cuyos datos son más variables y semi-estructurados.

Solución: **PostgreSQL** como motor relacional para todos los esquemas transaccionales (uno por microservicio, ADR-01), y **MongoDB** como motor documental exclusivamente para el esquema de Reportes dentro del servicio de administración.

Alternativa descartada: un único motor relacional (PostgreSQL) también para Reportes — se descartó para no forzar un esquema rígido sobre datos de reportes cuya estructura varía entre tipos de reporte, tal como ya anticipaba la sección de modelo de datos.

Consecuencias: cada esquema usa el motor más adecuado a su naturaleza (ACID para lo transaccional, esquema flexible para reportes), a costa de operar y respaldar dos motores de base de datos distintos en vez de uno solo.

ADR-07 — App móvil unificada para Cliente y Personal

ASR relacionados: ASR-08, ASR-10.

Problema: el diseño original (v1.1) proponía dos apps móviles nativas independientes, una por rol (Cliente y Personal). Al construir el prototipo de la Entrega 2 con un equipo pequeño, mantener dos proyectos Android desde cero duplica configuración de Gradle, tema visual y esqueleto de navegación sin que exista todavía una razón operativa concreta (distribución, permisos de dispositivo, política de actualización) que obligue a separarlos en esta etapa.

Solución: una única app (App/frontend/app-movil) con una pantalla de login compartida. Las credenciales identifican el rol del usuario autenticado y la navegación interna abre el conjunto de pantallas del rol correspondiente: Inicio/Entradas/Parqueadero/Pedidos para Cliente, Turnos/Asistencia/Validación para Personal. Esta decisión sigue vigente tras el cambio de stack de ADR-08: la app Flutter mantiene el mismo esquema de una sola app para ambos roles.

Alternativa descartada: mantener **App Móvil del Cliente** y **App Móvil del Personal** como dos módulos/APKs independientes (diseño de la v1.1) — se descartó para este prototipo porque el beneficio principal de separarlas (aislar del binario público de clientes las pantallas y permisos operativos del personal, y poder distribuirlas y versionarlas por canales distintos — tiendas públicas vs. MDM interno) todavía no tiene un requisito concreto que lo exija, y el costo de mantener dos proyectos no se justifica en esta fase.

Consecuencias: menos código y configuración duplicados mientras el equipo es pequeño, y un único punto de login para ambos roles. A cambio, el binario que instala un cliente cualquiera desde la tienda pública contiene también las pantallas y — potencialmente — los permisos operativos del personal (aunque inaccesibles sin credenciales válidas), lo que amplía la superficie de la app frente a la alternativa separada; y ambos roles quedan acoplados a un mismo ciclo de release. Si el rol Personal termina necesitando permisos de dispositivo, distribución o cadencia de actualización distintos a los del Cliente, esta decisión debe revisarse y volver a separar los binarios.

ADR-08 — Migración de la app móvil a Flutter

ASR relacionados: ASR-08, ASR-09, ASR-10.

Problema: ADR-05 fijó **Kotlin nativo** para la app móvil y descartó explícitamente los frameworks *cross-platform*, con el argumento de que compartir código de interfaz no aportaba un beneficio arquitectónico claro. Ese razonamiento omitió un requisito que sí es del proyecto: la app debe llegar a **Android y iOS**. Con el stack nativo, cubrir iOS obliga a escribir y mantener una segunda implementación completa (Swift/SwiftUI) de las mismas pantallas, lo que para un equipo de cuatro personas duplica el esfuerzo de toda evolución de la interfaz y multiplica el riesgo de que ambas plataformas se desincronicen.

Solución: la app móvil se reimplementa en **Flutter** (Dart), reemplazando la versión Kotlin/Jetpack Compose. Una sola base de código produce los binarios de Android e iOS, conservando el es-

quema de ADR-07 (una sola app para los roles Cliente y Personal, con el rol resuelto en el login). La implementación incluye un cliente de API centralizado (`lib/services/api_client.dart`) que concentra el consumo del backend, de modo que la futura integración con el API Gateway quede en un solo punto y no disperso por las pantallas (ASR-10).

Alternativas consideradas:

- **Mantener Kotlin y añadir una app iOS nativa en Swift/SwiftUI** — máxima fidelidad al look-and-feel de cada plataforma y acceso inmediato a APIs nuevas del sistema operativo, pero duplica la base de código de interfaz y el esfuerzo de mantenimiento (Modificabilidad, ASR-09).
- **Kotlin Multiplatform (KMM)** — comparte la lógica de negocio y de red pero mantiene la interfaz nativa por plataforma; es el término medio, y se descartó porque el grueso del trabajo de este prototipo está justamente en la interfaz, que KMM no comparte.
- **Flutter** (elegida) — una sola base de código para interfaz y lógica en ambas plataformas.

Consecuencias: una sola base de código para Android e iOS, y un único ciclo de build y pruebas para la app móvil, a costa de: (1) renunciar a los widgets nativos reales — Flutter dibuja su propia interfaz con su motor de render, de modo que la fidelidad con cada plataforma es una aproximación, no el control nativo; (2) sumar un tercer ecosistema de herramientas (Dart/pub) al del web (TypeScript/npm) y el del backend; y (3) una curva de aprendizaje en Dart para el equipo. La implementación Kotlin previa se elimina del repositorio; queda en el historial de Git si hiciera falta recuperarla.

Nota sobre la evidencia: esta decisión se tomó reimplementando la app completa en Flutter y comparándola con la versión Kotlin ya existente, no solo con un análisis en papel. La comparación sistemática de métricas entre ambas implementaciones (tiempo de desarrollo, líneas de código, arranque en frío, fidelidad visual) queda pendiente de documentarse como evidencia formal para la Entrega 1.

ADR-09 — Lenguaje y framework del backend

ASR relacionados: ASR-04, ASR-06, ASR-09, ASR-14.

Problema: `App/services/*` estaba marcado como “lenguaje por definir”. Es la única decisión que bloquea empezar a codificar los seis microservicios, y debía tomarse con evidencia medida — no por preferencia ni por popularidad.

Solución: los microservicios de dominio se implementan en **NestJS** (Node.js + TypeScript).

Alternativas consideradas: **NestJS** y **Spring Boot** (Java 17) se compararon construyendo el mismo endpoint en ambos y midiéndolos bajo carga (PoC-04, ver `App/PoCs/poc-04-lenguaje-backend/`). Se descartaron sin PoC: **Go**, porque nadie en el equipo lo conoce y sumar un lenguaje nuevo con la entrega encima es el mayor riesgo de cronograma; y **Python/FastAPI**, por su tipado opcional, poco conveniente con seis servicios desarrollados en paralelo por cuatro personas.

Evidencia (PoC-04): mismo endpoint, misma respuesta byte por byte, 100 conexiones concurrentes, 15 s por medición, cero errores:

Candidato	Escenario	req/s	p50 (ms)	p97,5 (ms)	RSS (MB)
NestJS	sin I/O	21.071	4	5	156
NestJS	con 2 ms de I/O	18.530	5	6	158
Spring Boot	sin I/O	91.441	0	1	335
Spring Boot	con 2 ms de I/O	32.288	3	4	337

Arranque hasta la primera respuesta: **NestJS 0,28 s** frente a **Spring Boot 1,09 s**.

Tres lecturas de esos números:

1. **Ambos cumplen RNF-07 con un margen enorme.** El umbral es $p95 \leq 500$ ms y el peor caso medido fue 7 ms: setenta veces por debajo. El rendimiento, por lo tanto, **no es el criterio que decide** esta elección — ninguno de los dos está cerca de incumplirlo.
2. **Spring Boot rinde más, y conviene decirlo sin matizarlo:** $4,3\times$ sin I/O. Pero la ventaja cae a $1,7\times$ al añadir 2 ms de I/O simulada, y contra una base de datos real (5–20 ms) se encogería más. Cuanto más se parece el escenario a producción, menos pesa la diferencia.
3. **NestJS arranca $4\times$ más rápido y usa menos de la mitad de memoria**, lo que favorece a RNF-15 (despliegue sin interrupción) y al escalado elástico de ASR-06.

Por qué se elige NestJS pese a que Spring Boot rinde más: porque el rendimiento ya está resuelto por ambos con setenta veces de margen, y los criterios que sí quedan abiertos favorecen a NestJS. En **Modificabilidad** (ASR-09) y **Comprobabilidad** (ASR-14, RNF-18), NestJS impone módulos e inyección de dependencias — el mismo modelo que el equipo ya aplica en los portales Angular, y el que hace que sustituir dependencias externas por dobles de prueba sea directo. Y el riesgo real del proyecto no es la latencia sino el cronograma: cuatro personas deben entregar seis microservicios, y usar el lenguaje que ya escriben en el frontend elimina un cambio de contexto completo.

Consecuencias: un solo lenguaje entre los portales web y el backend, con contratos de API que pueden generar tipos para ambos lados (App/shared/). A cambio se acepta un techo de rendimiento más bajo que el de Spring Boot — irrelevante frente a los umbrales actuales, pero que debería reevaluarse si algún servicio pasa a ser intensivo en CPU (los reportes con Map-Reduce, hoy delegados a MongoDB, son el candidato a vigilar). Node ejecuta en un solo hilo por proceso, de modo que el escalado es horizontal por réplicas, que es justamente lo que establecen ADR-01 y ASR-06.

Validez de la evidencia: el PoC mide un endpoint sin base de datos real, con cliente y servidor en la misma máquina, y no llegó a los 2.000 usuarios concurrentes de RNF-07 (con 100 conexiones el generador de carga ya era el cuello de botella). Tampoco mide lo que probablemente más pesa en la práctica: la velocidad de desarrollo del equipo. **Si alguno de los integrantes tuviera experiencia sólida en Java**, esta decisión sería genuinamente discutible y Spring Boot quedaría bien defendido por los datos.

ADR-10 — Motor de la cola de mensajes: RabbitMQ

ASR relacionados: ASR-04, ASR-07, ASR-13.

Problema: ADR-04 estableció que hace falta una cola de mensajes, pero dejó el motor sin decidir, escrito como “RabbitMQ/Kafka” — como si fueran intercambiables. No lo son: responden a modelos distintos (cola de tareas frente a registro de eventos) y la elección condiciona cómo se escriben los consumidores en los seis microservicios.

Solución: RabbitMQ.

Evidencia (PoC-05, App/PoCs/poc-05-cola-mensajes/): se publicó y consumió el evento real ENTRADA_TRANSFERIDA en ambos brokers (los dos en Docker, 500 mensajes por combinación), midiendo la latencia de extremo a extremo:

Broker	Modo	p50 (ms)	p95 (ms)	msg/s
RabbitMQ	rápido (sin persistir)	16,3	17,5	18.932
Kafka	rápido (acks=0)	32,1	36,0	12.418
RabbitMQ	duradero (confirmado)	162,0	282,1	1.619
Kafka	duradero (acks=all)	31,6	57,9	7.702

El ganador se invierte según el modo, y conviene decirlo sin matizarlo: sin durabilidad RabbitMQ es el doble de rápido, pero **con durabilidad Kafka es cinco veces mejor** (31,6 ms frente a 162 ms de mediana). La razón es estructural: RabbitMQ hace `fsync` por mensaje al persistir y su latencia se multiplica por diez, mientras que Kafka escribe a un log secuencial y la durabilidad le sale prácticamente gratis. Y el modo que importa aquí es el duradero: ni una transferencia de entrada ni una alerta de evacuación pueden perderse porque el broker se reinició.

Sin embargo, **ambos cumplen RNF-17 con enorme margen**: el umbral es 10 s y el peor caso medido —RabbitMQ duradero, p95— fue **282 ms**, treinta y cinco veces por debajo. Igual que en ADR-09, la latencia no es el criterio que decide.

Se midió entonces lo que sí queda abierto, con el mismo escenario en ambos (10 eventos, 3 rechazados a propósito):

	RabbitMQ	Kafka
Cola de mensajes muertos (DLQ)	nativa, 5 líneas declarativas	no existe: hay que construirla
Imagen / memoria en reposo	275 MB / 147 MB	634 MB / 309 MB
Líneas para levantarlo	9	15
Consola de administración	incluida	no incluida
Conceptos a dominar	colas, <i>exchanges</i> , <i>bindings</i>	+ particiones, offsets, <i>consumer groups</i> , ret

Por qué RabbitMQ pese a que Kafka es más rápido en modo duradero: porque el uso que este sistema le da a la cola es el de una **cola de tareas** —disparar una notificación, generar un QR, registrar una auditoría, avisar de una evacuación— y no el de un *registro de eventos* con reproducción histórica y múltiples consumidores independientes, que es donde Kafka se justifica. Para ese uso, RabbitMQ ofrece de fábrica lo que aquí sí hace falta: **DLQ y reintentos nativos**,

que ASR-07 y ASR-13 exigen para no perder en silencio un evento que falló. En Kafka eso hay que construirlo y mantenerlo a mano. A eso se suma la mitad de memoria, la mitad de configuración y una consola incluida, con un equipo de cuatro personas que además debe operar PostgreSQL, MongoDB y Redis.

Consecuencias: se acepta una latencia mayor en modo duradero (282 ms de p95 frente a los 58 ms de Kafka) — irrelevante frente al umbral de 10 s de RNF-17 — y un techo de throughput más bajo (1.619 msg/s duraderos). Ese techo es holgado para el volumen del sistema: los eventos se generan por transferencia, pedido o alerta, no de forma continua. **Cuándo habría que revisar esta decisión:** si aparece la necesidad de reproducir el histórico de eventos, de que varios consumidores independientes lean el mismo flujo, o si el volumen se acerca al millar de mensajes por segundo sostenido. En cualquiera de esos casos Kafka pasa a estar mejor justificado, y los números de este PoC lo respaldarían.

Validez de la evidencia: un solo nodo de cada broker (en producción irían replicados, y ahí Kafka replica mejor), todo en la misma máquina sin red de por medio, y 500 mensajes por medición — no se probó el comportamiento con colas de millones de mensajes, que es donde Kafka se separa de verdad. Tampoco se midió la recuperación ante caída del broker, que es justamente el escenario que motiva el modo duradero.

ADR-11 — Autenticación entre servicios: RS256 y dos tokens

Contexto: RNF-06 exige que *toda* petición a un microservicio llegue autenticada y autorizada por rol. Con seis dominios independientes (ADR-01), la pregunta es quién valida la sesión y con qué. El CU-027 añade dos exigencias propias: que cerrar sesión surta efecto de verdad — un JWT es válido hasta que expira, lo firme quien lo firme — y que la sesión no se caiga sola mientras la persona está usando la aplicación.

Alternativas consideradas:

- **HS256 con un secreto compartido.** Fue la primera implementación. Funciona mientras un solo servicio verifica, pero con HS256 firmar y verificar usan la misma clave: cada servicio que valida tokens podría **fabricar uno de administrador**. Con seis microservicios, seis lugares por donde se filtra la llave maestra.
- **Consultar al servicio de Administración en cada petición.** Autorización siempre exacta, pero convierte a ese servicio en un punto único de fallo para todo el sistema y le suma una llamada de red a cada operación.
- **RS256 con clave asimétrica** — la elegida.

Decisión: tokens JWT firmados con **RS256**. La clave **privada** la tiene solo el servicio de Administración, que es el único que emite; los demás reciben la **pública**, que sirve para verificar y no para firmar. Al arrancar se comprueba que las dos claves se correspondan, que sean RSA de 2048 bits o más y que en producción no sean las de desarrollo del repositorio.

Dos decisiones acompañan a la principal:

- **Revocación por Redis (ADR-03).** Cada sesión cerrada se publica como `sesion-revocada:<jti>`

con una vida igual a lo que le quedaba al token. Así los demás servicios rechazan un token revocado sin preguntar a Administración, y la lista no crece: cuando la clave expira, el token ya no valía por sí solo.

- **Dos tokens.** Uno de **acceso** de 15 minutos, que viaja en cada petición, y uno de **renovación** de 30 días sin uso, que solo viaja al renovar. Cada renovación emite uno nuevo e invalida el anterior; reutilizar uno gastado cierra la sesión entera, porque significa que dos partes tienen copia. En el navegador el token de renovación va en una cookie `HttpOnly; SameSite=Strict`, fuera del alcance de cualquier script inyectado.

Atributo de calidad: Seguridad (ASR-03) como motivación principal; Disponibilidad (ASR-02), porque validar sin consultar evita el punto único de fallo; y Usabilidad (ASR-08), porque los dos tokens evitan que la sesión se cierre sola con la aplicación abierta.

Evidencia: implementado y probado sobre el sistema real. Las pruebas de `App/services/*/pruebas/` comprueban que un token con `alg: none` o firmado con la clave pública como secreto HMAC se rechaza; que una sesión cerrada deja de valer *en el otro microservicio*; y que reutilizar un token de renovación ya gastado cierra la sesión. El API Gateway (ADR-02) valida por subpetición contra `/sesiones/verificar`, y una prueba de defensa en profundidad confirma que llamar directo a un servicio con la cabecera de identidad falsificada devuelve 401.

Consecuencias: firmar con RSA cuesta más CPU que con HMAC, y cada petición protegida añade una subpetición interna del gateway. A cambio, ningún servicio distinto del emisor puede fabricar un token. **Cuándo revisar:** si el coste de la subpetición se notara bajo carga, los servicios pueden verificar la firma por su cuenta con la clave pública y consultar Redis directamente — sigue sin necesitar la privada.

15. Riesgos técnicos

Riesgo	Probabilidad	Impacto	Prioridad	Plan de mitigación	Plan de contingencia
Concurrencia entre compradores en el mercado secundario	Media	Alto	Alta	Bloqueo distribuido con TTL en Redis antes de procesar el pago.	Rechazar la segunda transacción y notificar de inmediato al comprador no atendido.
Caída del API Gateway como punto único de entrada	Baja	Alto	Alta	Múltiples instancias + balanceador de carga + <i>health checks</i> .	Failover automático hacia una instancia o región de respaldo.

Riesgo	Probabili- dad	Impacto	Priori- dad	Plan de miti- gación	Plan de con- tingencia
Indisponibilidad de la pasarela de pagos externa	Media	Alto	Alta	<i>Timeouts</i> , reintentos y cola de reconciliación de pagos.	Marcar la transacción como pendiente y permitir reintento manual del usuario.
Picos de tráfico en apertura de venta / ingreso masivo	Alta	Medio	Alta	Autoescalado del servicio de Entradas + caché en Redis.	Sala de espera virtual (cola de acceso) para limitar la concurrencia admitida.
Fallo del proveedor de notificaciones durante una emergencia	Baja	Alto	Media	Reintentos automáticos + canal secundario (SMS si falla push).	Escalar el aviso de forma manual mediante el personal en sitio.
Pérdida de conexión de una app móvil durante una evacuación	Media	Medio	Media	Caché local de instrucciones + reconexión automática.	El personal de seguridad releva la comunicación directamente en sitio.

16. Glosario

- **SAD:** Descripción de la Arquitectura del Software (*Software Architecture Description*), este documento.
- **ASR:** Requisito Arquitectónicamente Significativo (*Architecturally Significant Requirement*).
- **ADR:** Registro de Decisión Arquitectónica (*Architecture Decision Record*).
- **CU:** Caso de Uso.
- **QA:** Atributo de Calidad (*Quality Attribute*).
- **Modelo C4:** notación de Simon Brown para diagramar arquitectura en niveles (Contexto, Contenedores, Componentes, Código).
- **API Gateway:** componente que centraliza el enrutamiento, la autenticación y la autorización de las solicitudes al backend.

- **Microservicio:** servicio backend independiente, responsable de un dominio de negocio, con su propia base de datos.
- **Redis:** base de datos en memoria usada como caché y para bloqueos distribuidos.
- **RabbitMQ / Kafka:** sistemas de colas de mensajes usados para procesamiento asíncrono entre servicios.
- **AMQP:** protocolo de mensajería usado por colas como RabbitMQ.
- **QR:** código de respuesta rápida (*Quick Response*) usado para identificar entradas y validar pedidos.
- **Aforo:** cantidad de personas presentes o permitidas en un recinto o zona.
- **Mercado secundario:** reventa de entradas ya adquiridas entre usuarios del sistema.
- **CDN:** red de distribución de contenido (*Content Delivery Network*) usada para servir archivos estáticos cerca del usuario.
- **Balanceador de carga:** componente que distribuye solicitudes entre varias instancias de un servicio.
- **Bloqueo distribuido:** mecanismo (implementado aquí con Redis) que impide que dos procesos modifiquen simultáneamente el mismo recurso.
- **Consistencia eventual:** garantía de que los datos de distintos servicios convergen al mismo estado, aunque no de forma instantánea.

17. Referencias

- arc42. *arc42 overview*. <https://arc42.org/overview>
- Brown, S. *The C4 model for visualising software architecture*. <https://c4model.com>
- Brown, S. *Software architecture diagram review checklist*. <https://c4model.com/diagrams/checklist>
- Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley. (Base teórica del Árbol de Utilidad y los ADR usados en este documento.)
- Documentos internos del proyecto: *ArchitecturalProposal.tex*, *C4Diagrams.tex*, *Submission/DistribucionCasosUso.pdf* y *BitacoraArquitectonica.md* (equipo Hexacore, 2026).